



图 1: Black Hat USA 2026 主题演讲《The End of Rare Defending When Offense Is Cheap》(YouTube 视频 uNdlqEqxUVw) 封面: 当攻击因 AI 自动化而变得廉价时, 基于攻击稀缺的稀有防御时代正在结束, 防御者必须改写攻防的物理规律。

Source (source_timestamp): 00:00:00–00:00:16

1 总论 | 结论: 稀有防御的时代正在结束, 防御者必须改写攻防物理规律; 机制: 安全建立在”攻击稀缺”前提上, AI 正把攻击变成廉价商品; 证据: 漏洞与利用生成曲线; 读者所得: 建立”前提 → 崩塌 → 转机 → 行动”的判定地图

1.1 动机: 从一段“虚拟入侵”新闻短片说起

本讲是 Black Hat USA 2026 大会第二天主舞台的主题演讲。开场约 00:00:17–00:00:39 先播放了一段新闻蒙太奇: 关于一场大规模网络攻击的警告, 画面称美国正“遭受虚拟入侵”(America under virtual invasion)。演讲者有意用攻击威胁的实感而非抽象的行业趋势开场, 把听众立刻带回一个问题: 当对手的进攻手段越来越廉价, 以“稀缺”为前提的整个防御体系还站得住吗?

随后, Black Hat 总裁 Susie Pallet 于 00:01:25–00:04:40 致欢迎辞, 向在场者发出“不要只是参会, 要去发现”(Don't just attend, discover) 的挑战; 大会创始人 Jeff Moss 在 00:04:40–00:14:20 回忆本届大会的四大主题——AI 与自主威胁、网络冲突与真实作战、信息作战背景下的工程韧性, 以及身份、信任与控制。Jeff Moss 特别点出: 安全已成为一种政治议题, 防御者不能置身事外。这些铺垫共同说明, 这场主题演讲被安排在“防御”一侧, 讨论的是面对 AI 时代攻击生产力暴涨, 防御应当如何重新设计。

1.2 主张:当攻击不再稀缺,防御必须改写物理规律

00:13:54–00:14:20 的引介告诉我们演讲者是谁: Dave Wesson 现负责微软 AI 安全团队,为大规模防御构建 AI 模型、agent 与评估系统。他登台后第一句话就交代了整场演讲的底色——在满屋“AI 末日论”的氛围里,他可能是全场唯一的乐观主义者,并自嘲要把这场演讲当作一条“30 分钟的高强度社交媒体动态”(00:14:47–00:15:04),因为听众无法取关他。

原声片段

演讲者 (00:14:58–00:15:07): “I’m going to talk to you about what I think happens when attacks get less rare, less scarce, and what we can do to defend in that environment.” ——我要和你们谈谈,当攻击不再稀有、不再稀缺时会发生什么,以及我们该如何在这种环境中防御。

这句话是全文的总契约:攻击正在变得不稀缺,防御必须随之重设计。这里引入本讲两个核心术语。**rare defending** (稀有防御)指整个网络安全产业基于“安全边界极难被击穿、攻击稀缺”这一未言明前提来设计控制的防御范式:边界(网络、进程、身份、加密)一旦失守即灾难,因此防御者把大部分资源押在边界本身。与之相对的是本讲的核心判断 **offense is cheap** (攻击成本低廉):当 AI 把漏洞发现、利用生成与规避检测都变成量产商品时,稀缺不再保护防御方。演讲者随后在 00:15:48–00:16:07 把这条假设说得更直白:安全有一个未言明的前提——安全边界极其难以击穿,因此击穿它们的攻击是稀缺的。他在 00:16:12–00:16:43 进一步把整个网络安全比作一座房子:边界用来隔离网络、在云端分离信任域、建立认证加密与数据保护策略、用 hypervisor 或进程沙箱遏制失效;房子从地基到屋顶都建立在“这些边界不会轻易被击穿”之上。本节先记下这个前提,后文 00:15:48–00:18:13 将展开其经济学,00:20:42–00:22:17 起将用数据检验它是否仍在成立。

1.3 机制:站在攻防两侧的双重视角

为什么这场演讲值得信? 00:15:07–00:15:40 演讲者交代的自己的来源。他在安全一线摸爬滚打二十年:经历过 WannaCry、Stuxnet 这类事件,为 Windows、Linux、Azure 构建操作系统安全,做过 EDR,带过漏洞红队;而最近九个月他“投奔了黑暗面”——转而开发漏洞挖掘 harness、训练前沿模型 (frontier model) 的网络安全能力,同时继续构建防御。

背景知识

这个“攻防双侧”的身份是理解全文叙述可信度的关键。演讲者不是纸上谈兵的行业观察者:他既亲眼看到漏洞挖掘 harness 与自动化利用生成把攻击成本压到何等低位,也掌握微软内部关于防御工具(内存安全迁移、形式化方法、预防性 agent)的一手数据。全文反复出现的“同一个生产力优势”(the same productivity advantage)叙事,正是建立在这双重视角之上——攻击者用它造漏洞,防御者也可以用它造安全。

00:15:41–00:15:48 他自述希望把这两个观察视角结合起来,谈一谈他认为的未来走向。这意味着读者可以把本讲理解成一份“攻防两侧同源生产力”的说明书,而不是单纯的安全威胁报告。

1.4 证据：一条“前提 → 崩塌 → 转机 → 行动”的判定地图

为了让读者带着地图阅读全文，本节先给出整场演讲的论证骨架。其一，**稀缺前提**：网络安全建立在“边界难击穿、攻击稀缺”之上，漏洞赏金定价、每年约 90 个野外利用、检测依赖不变性，都是这一前提的产物（后文 00:15:48–00:20:40 展开）。其二，**崩塌证据**：MSRC 漏洞处理量九倍于三月、内部 harness 贡献全年 66% 的关键高危问题、182 个自动生成的崩溃级 POC、Exploit Gym 上 157/898 的利用生成率，都在说同一件事——攻击已变便宜且趋势不可逆（后文 00:20:42–00:29:43 展开）。其三，**转机**：防御者不应与攻击者“肉搏”，而应把同一生产力优势投入持久性——**secure by construction**、形式化方法与预防性 agent，用改变攻防物理规律来扭转经济学（后文 00:29:43 起展开）。其四，**行动**：在迁移完成之前，今天就有三件可以立即做的事。

核心结论

本讲的判定地图：**前提**（攻击稀缺是旧范式的未言明地基）→ **崩塌**（AI 让漏洞发现、利用生成与规避检测都成为量产商品，数据不支持稀缺假设继续成立）→ **转机**（不与攻击者肉搏，而是用同一生产力优势改写攻防物理规律：持久性、预防、确定性安全）→ **行动**（最高风险面 **secure by design**、关键边界形式化验证、基础设施即代码与攻击图支撑的预防性 agent 军队）。全文六个章节都按这张地图推进。

边界与风险

不要把“乐观”误解为“无所谓”。演讲者的乐观建立在明确的前提上：只有当防御方把生产力优势投入持久性与确定性安全，而不是与攻击者比谁更快地“漏洞换补丁、利用换检测”，才谈得上扭转局势。若继续沿旧范式肉搏，防御方将因天然的不对称劣势而失败。

1.5 本章小结

本章建立了全文的总契约与读者路径。**教学目标**：读者完成全文后，应能解释为什么“攻击稀缺”这一前提正在崩塌，并能用“改变物理规律”（持久性、预防、确定性安全）而不是“与攻击者肉搏”来回应自动化漏洞挖掘与利用生成。**读者路径**：**section_02–03** 依次掌握“稀缺前提”及其崩塌证据，**section_04–05** 掌握“改变物理规律”的两条机制（构建即安全与形式化方法），**section_06** 把机制压缩为三件今天可做的事。后续章节将沿着“前提 → 崩塌 → 转机 → 行动”的弧线，先审视那个未言明的假设（稀有防御），再用数据检验它是否仍然成立。

2 稀缺前提 | 结论:整个网络安全是建立在“边界难击穿、攻击稀缺”之上的稀有防御;机制:安全边界与漏洞、利用、规避检测的分层定价经济学;证据:漏洞赏金定价、每年约 90 个野外利用、两种由此推导的防御策略;读者所得:识别自己依赖了哪些稀缺假设

2.1 结论:网络安全的地基是一条未言明的假设

上一章建立了全文总契约:攻击正在变得不稀缺,防御必须随之重设计。本章回到问题的起点,仔细解剖旧范式的地基。00:15:48–00:16:07 演讲者给出关键断言:安全产业有一个未言明的前提(underlying assumption)——我们拥有网络、进程、身份、加密这些 **security boundary** (安全边界),它们极其难以击穿,因此击穿它们的攻击是稀缺的。他在 00:16:04–00:16:12 抛出一个问题:如果这个前提变了,会怎样?

核心结论

整个网络安全——演讲者称之为我们居住的“网络房子”,从屋顶到地基——都建立在“边界很难被击穿、攻击很稀缺”这一前提之上。边界被用来隔离网络、在云端分离信任域、建立认证加密与数据保护策略、用 **hypervisor** 或进程沙箱遏制失效;公司、业务、手机上的每一个控制与策略,都依赖这些边界“不容易被击穿”(00:16:12–00:16:43)。这就是 **rare defending** (稀有防御) 范式:把大部分资源押在边界本身,默认边界失守是小概率事件。

2.2 机制:安全边界、漏洞与利用的分层定价经济学

稀缺不是一句口号,而是体现在明码标价里。00:16:47–00:17:16 演讲者以漏洞赏金(bug bounty)为例说明:击穿某些边界“没有便宜的价格”,而定价随边界的重要性递增——如果要在公开市场上买一个 **hypervisor bypass**,价格大约是一个 **process** 边界绕过的 20 倍。**hypervisor** 遏制整个虚拟机的失效, **process** 边界只保护单个进程,重要性不同,稀缺价格便不同。这一价格体系已经存在很久,并被整个行业视为对现实的自然反映。

00:17:16–00:18:13 演讲者再引入一对术语,把“稀缺”从边界推广到攻击链条的每一环。**potential risk / actualized risk** (潜在风险 / 已实现风险):漏洞(vulnerability)只是潜在风险;利用(exploitation)与已实现的攻击(implemented attack)才是已实现风险,后者因计入专业性与稀缺性(例如绕过缓解措施的技巧)而远为昂贵。漏洞与利用在市场上的标价差异正是这种定价的体现:vulnerability 便宜,exploit 昂贵。稀缺性经济学(scarcity economics)由此贯穿整条价值链——边界、漏洞、利用、规避检测,每一环的昂贵都源于供给不足。

2.3 证据:90 个野外利用与两种衍生防御策略

定价模型与野外数据互为印证。00:17:38–00:18:05 演讲者给出一个惊人比例:尽管每天在 LinkedIn 和新闻上看到数以万计的 CVE,由 Google 团队追踪的每年真实发生的野外利用(in-the-wild exploits)只有约 90 个。对一个边界而言,被击穿在这个年代几乎是“罕见事件”。00:18:05–00:18:13

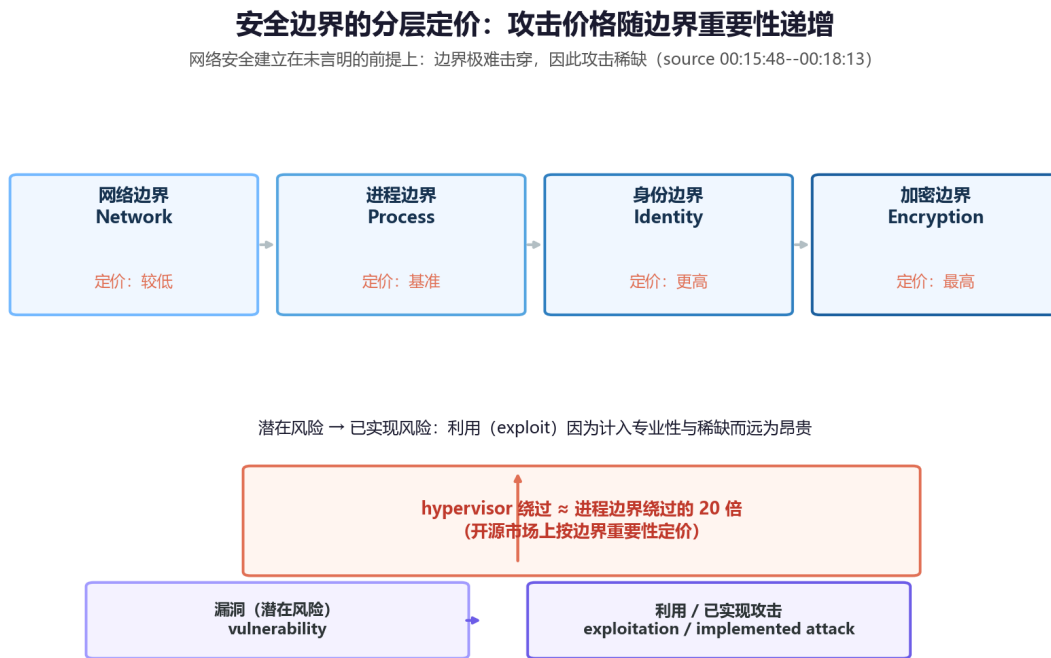


图 2: 安全边界的稀缺定价: 网络、进程、身份、加密四类边界的攻击价格随重要性递增, hypervisor 绕过约为进程边界绕过的 20 倍; 漏洞 (潜在风险) 与利用 (已实现风险) 的价格因专业性与稀缺而进一步拉大, 整个网络安全建立在攻击稀缺的供给经济学前提之上。

Source (source_timestamp): 00:15:48–00:18:13

由此得出结论: 整个网络安全的根基, 就是这种围绕稀缺与供给的供应经济学 (supply economics)。

而这条前提确实“长久以来都成立”, 并得到业界报告的支持: 00:18:13–00:18:53 指出, 今天的大多数入侵事件发生在边界之上而非击穿边界——Verizon 泄露报告显示大部分攻击落在凭据窃取层面, 传统上比击穿边界便宜得多, 也经由钓鱼与社会工程完成; 而当某个漏洞真的被利用时, 绝大多数是**已知漏洞**, 意味着攻击者有充足时间实现利用。所有经济学与战略都建筑在此之上。

在此基础上可以推导出两种完全依赖稀缺前提的**主导防御策略**。第一种是弱化或降低 SDLC 的优先级 (00:18:53–00:19:43): 既然漏洞很少被利用, 静态分析、更安全的语言、最小权限、强身份都可以让位于“快速打补丁”。演讲者给出的压力测试是: 如果利用变成另一种大宗商品会怎样? 90 年代的经验已经证明“漏洞好利用就快速打补丁”的策略携带显著风险。第二种是减少预防投入、转向“检测并快速响应” (00:19:41–00:20:28): 这是所有厂商都在推销的方案——撒一点 AI, 就能检测并响应。但真相是 **dwell time** (驻留时间)——攻击者进入环境到被发现之间的持续时间——正在变长, 检测越来越难。检测本质上依赖“不会变的不变量” (invariants): 攻击者也是软件开发者, 他们付不起在每个作战行动里更换软件、植入体、C2 与横向移动工具的代价。可一旦这些变得便宜, 供应经济学也随之改变。

边界与风险

两种主导策略都建立在“攻击稀缺”之上, 且都指向同一个压力点: 若攻击变成量产商品, 快速补丁将跟不上利用生成的速度, 而基于不变量的检测将被“每次行动重新生成工具与框架”的对手绕过。识别自己的防线依赖了哪些稀缺假设, 是评估暴露面的第一步。

2.4 读者所得：识别自己依赖了哪些稀缺假设

本章把“稀有防御”的具体内容拆开来看：网络、进程、身份、加密四类边界按重要性分层定价（hypervisor bypass 约为 process 的 20 倍，00:16:47–00:17:16）；漏洞是潜在风险、利用是已实现风险，后者因专业性与稀缺而昂贵（00:17:16–00:18:13）；每年野外利用仅约 90 个，边界被击穿属于例外而非常态（00:17:38–00:18:05）；由此衍生出“弱化 SDLC、快速补丁”与“少预防、检测响应”两种策略（00:18:53–00:20:28）。

背景知识

读者可据此给自己画一张“稀缺依赖清单”：你的基础设施把多少安全押在“边界不会轻易被击穿”？你的响应计划假设攻击者多久更换一次工具与 C2？你的检测规则有多少条依赖不变量（hash、IP、域、TTP）保持稳定？每一项依赖，都是后文“崩塌证据”章节要检验的对象。

本章小结：稀缺性经济学是旧范式的定价系统，稀有防御是它的建筑形式。00:20:28–00:20:40 演讲者抛出本章最后的追问：当稀缺原则不再站在我们这边时，防御会是什么样？我们真的已经走到那一步了吗？下一章用数据回答这个问题。

3 崩塌证据 | 结论：攻击已变便宜且趋势不可逆，限制模型访问无法恢复稀缺；机制：漏洞量曲线、自动化利用生成、检测不变性失效与自主操作轨迹；证据：MSRC 九月环比九倍、66% 关键问题、182 个自动 POC、Exploit Gym 157/900、Anthropic 与 Drago 案例；读者所得：能用数据复核”攻击稀缺”假设是否仍然成立

3.1 结论：稀缺前提正在被数据瓦解

上一章识别的稀缺假设是“边界难击穿、攻击稀缺”。本章直接检验它：00:20:35–00:20:42 演讲者宣布，漏洞稀缺这一前提——连同“漏洞只是潜在风险、利用仍昂贵”的推论——“正在被瓦解”（being undermined as we speak）。00:20:28–00:20:40 的问题“我们真的已经走到那一步了吗”将在本章用四组证据回答：漏洞量曲线、自动化利用生成、检测不变性的失效，以及自主操作的能力轨迹。四组证据指向同一个结论：攻击已变便宜且趋势不可逆，限制模型访问无法恢复稀缺。

3.2 机制与证据（一）：漏洞量曲线——MSRC 九月环比九倍

00:20:42–00:22:17 给出了第一组数字。这是本年度 AI 给厂商施加压力的“漏洞量曲线”（vulnerability curve）：MSRC 每约六周就把处理与修复的漏洞数量翻一番，当前漏洞处理量已经是三月份的九倍。演讲者说明这是微软的数据，但趋势在 Google、Apple 等主流软件厂商中同样成立；他也坦诚尚不知道曲线是否持续，但若持续下去，所有“假定漏洞稀缺”的地方都将陷入困境。该数据覆盖微软使用的开源组件与 Windows、Office 等第一方软件（00:21:28–00:21:46）。

曲线与 AI 高度相关还是只是人类更会找 bug？00:21:46–00:22:17 的叙述给出内部证据：微软

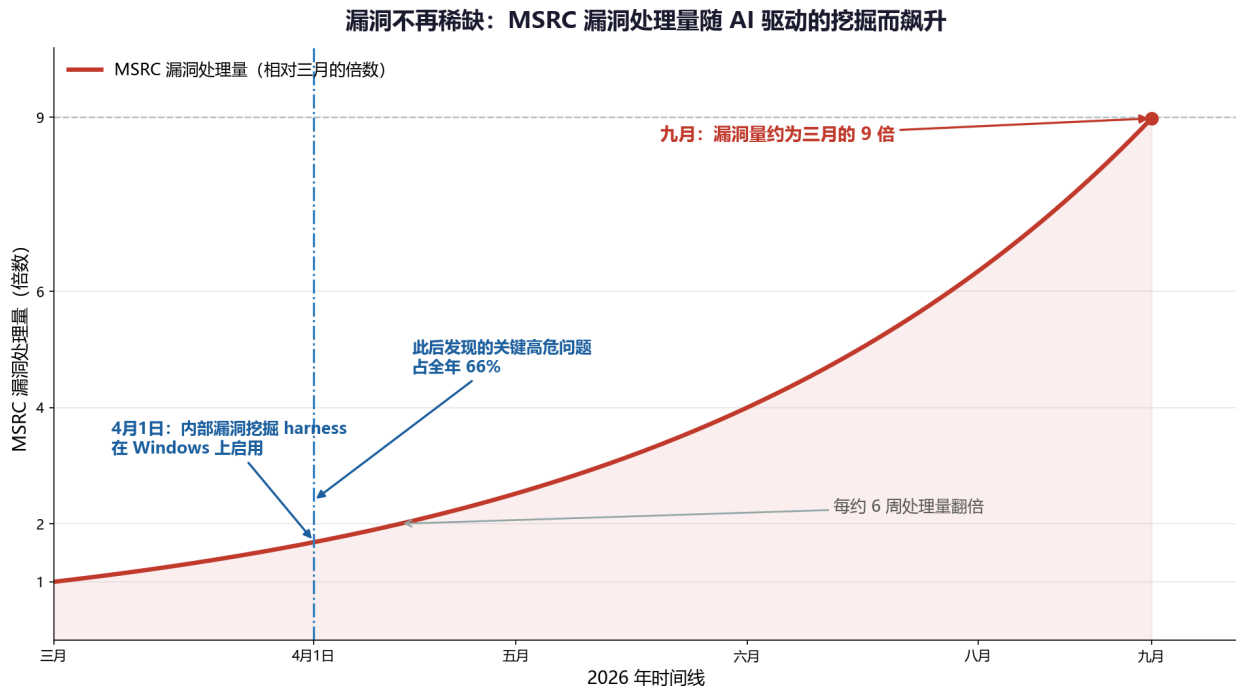


图 3: 漏洞不再稀缺：MSRC 漏洞处理量每约 6 周翻倍、九月已达三月的 9 倍；4 月 1 日在 Windows 上启用的内部漏洞挖掘 harness 贡献了全年 66% 的关键高危问题，证明漏洞稀缺前提正被 AI 驱动的挖掘所瓦解。

Source (source_timestamp): 00:20:42–00:22:17

在四月一日于 Windows 上启用了一个新的内部 **vulnerability discovery harness**（漏洞挖掘 harness）——一种包裹模型的运行框架，通过注入上下文、工具与校验逻辑放大模型发现漏洞的能力。启用之后，harness 找到的关键高危问题数量占全年该级别问题的 **66%**。“这不是巧合，这是事实：是 AI 在驱动这一切。”曲线上的漏洞是真实的边界级漏洞：仅一个数据集里就有 7 个同时跨越内核边界与远程边界的 TCP/IP 漏洞，对 Azure 与全球每一台 Windows 系统都至关重要；这类漏洞过去要靠专家定制一年，现在正以工业速度被“喷出来”（00:22:17–00:22:49）。且不限于 Windows——Linux 与其他操作系统的数据同样强相关（00:22:49–00:22:56）。

3.3 机制与证据（二）：自动化利用生成——182 个自动 POC 与 Exploit Gym

潜在风险暴涨之后，已实现风险（利用）呢？00:23:02–00:24:16 给出第二组数字。微软内部漏洞挖掘 harness **Mdash** 在 Azure 的 Linux 发行版中发现了约 200 个 Linux 内核漏洞；新加入的模块能把静态分析结果直接转换成 POC。结果远超预期：**200 个漏洞中自动生成了 182 个崩溃级 POC**，其中许多是完整可用的利用——包括从漏洞自动“吐”出的 root 级利用。平均 token 成本 **3.61 美元**，平均耗时 **21 分钟**。演讲者提醒：世界大部分计算都跑在某种 Linux 上，依赖内核边界保命，而这条边界正在被攻击。

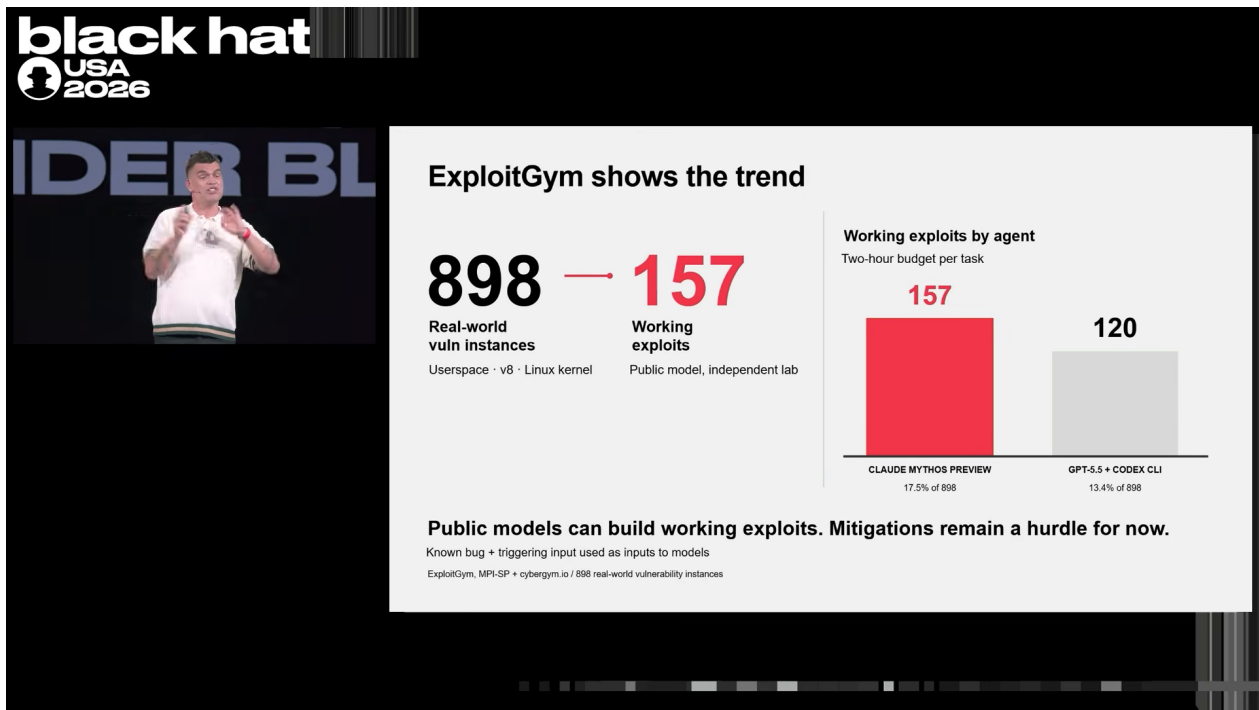


图 4: Exploit Gym 显示趋势: 对 898 个真实漏洞 (涵盖用户态、V8 与 Linux 内核), 公开模型以两小时任务预算自动生成 157 个可工作利用 (Claude Mythos 预览 157 个、GPT-5.5 + Codex 120 个); “公开模型已能构建可工作的利用, 缓解措施目前仍是障碍”, 说明漏洞发现 harness 把静态分析自动转为 POC 已使攻击稀缺前提崩塌。

Source (source_timestamp): 00:23:09–00:24:40

原声片段

演讲者 (00:23:41–00:24:16): 对约 200 个漏洞, “we can automatically generate 182 crash level POCs. Many of them are fully working exploits. I’m talking root exploits automatically spit out from a vulnerability. And the average cost from a token perspective, \$3.61, 21 minutes on average.” ——我们能自动生成 182 个崩溃级 POC, 其中许多是完整可用的利用, 我是说从漏洞自动吐出 root 级利用, 平均 token 成本 3.61 美元、平均 21 分钟。

这不仅是内部数据。00:24:13–00:24:40 提到公开基准 **Exploit Gym**: 前沿模型正在飞速进步, 演讲者早上看的时候, OpenAI 已把某条记录的数字大约翻了一倍。在 898 个真实世界漏洞中 (不只是相对易利用的 Linux 内核漏洞, 还包括浏览器漏洞等), 模型能生成 157 个利用。当前真正拖后腿的是非确定性缓解 (ASLR、控制流随机化等): 它们让利用更难, 但无法保证不被利用, 因此 “不要赌这条曲线会掉头” (00:24:40–00:24:59)。00:24:59–00:25:12 的预测更直接: 到年底, 自动化利用生成 (automatic exploit generation) 将成为常态与大宗商品——把静态分析结果或漏洞描述直接转换为可工作的利用, 届时不再是前沿能力, 而是量产供给。

3.4 机制与证据 (三): 限制模型访问无法恢复稀缺

“这些都是高级前沿模型, 我们正在限制它们, 限制访问难道不能维持稀缺、救我们吗?” 00:25:12–00:26:34 的回答斩钉截铁: 不能。依据是 CyberJ 基准榜单: 顶端的参赛者并不是前沿模型本身, 而



图 5: 痛苦金字塔 (David Bianco, 2013): 大多数检测建立在“制品复用”的不变性假设上——hash、IP、域名易变,而工具与制品昂贵难改;但自主操作把“混淆”变成“按目标生成”、把“编辑工具”变成“按需写码”,使“规避检测稀缺”的前提失效。

Source (source_timestamp): 00:27:13–00:28:00

是 **harness** (能力注入框架)。**capability harness** (能力注入框架)强调:网络安全知识不必然住在模型权重里,可以经工具、上下文甚至 markdown 文件注入 **agent**; **harness** 使用前沿模型,但同时把网络专家知识通过工具编码进去,而且在很多情况下 markdown 注入“更优”。结论是:用策略与访问限制来“限制出稀缺”是不现实的,多个厂商在 **Cyber Gym** 上的 **harness** 就是当前最强证据。00:26:34–00:27:13 进一步指出,检测假设同样不成立——先入之见是“编写框架或植入体极其昂贵,所以攻击者只能反复使用加壳器与混淆工具、沿用同一套 **TTP**”,由此获得检测的持久性;但这等于默认“规避检测”是稀缺属性。

3.5 机制与证据 (四): 检测不变性失效与自主操作轨迹

检测依赖什么? 00:27:13–00:27:37 引入 **pyramid of pain** (痛苦金字塔)——描述检测所依赖的不变性模型: hash、IP、域便宜易变,而 **TTP**、工具、制品昂贵难改,因此检测可以长期有效。但 00:27:37–00:27:57 的转折是本章的关键机制:过去攻击者要“重新训练操作员”才换得了工具,代价高昂;如今改用**自主操作 (autonomous operations)**,不再需要混淆,而是按目标逐个定制专用工具与框架,无需更换操作员。这直接击穿金字塔底层的“制品不常变”假设,规避检测不再稀缺。

三组真实案例支撑这一判断。其一, **Anthropic** 去年十一月报告:一个网络操作员对顶级目标的大部分作战行动——约 **80–90%**——由云端代码配合子 **agent** 完成,结果“看起来相当好”(00:27:57–00:28:29)。其二, **Drago** 今年五月报告水厂事件:一个 **AI** 辅助组织在作战过程中实时构建框架,代码不断重新生成、带满 **AI** 生成的痕迹;仅这一次行动就生成了 **17000 行 C2 植入体代码**(00:28:25–00:29:04)——这就是“制品保持不变的假设已经失效”的直接证明。其三,轨迹数据: **UK ISEI** (英

国某安全研究所)测试前沿模型执行 32 步自主入侵作战的能力,当前在 1000 万 token 预算内能完成 9.8 步,且仅今年一年就提升 59% (00:29:04–00:29:32)。00:29:32–00:29:43 的结论是:自主操作将成为常态,不要假设只有特定模型才能做到——能力可以注入任何位置,稀缺不会来自限制。

核心结论

四组证据合起来回答上一章的问题:攻击已变便宜且趋势不可逆——漏洞量曲线九倍于三月、66% 关键问题来自内部 harness、200 个漏洞自动产出 182 个崩溃级 POC (3.61 美元、21 分钟)、Exploit Gym 157/898 且记录仍在翻倍、80–90% 的作战由自主代码完成、32 步入侵完成 9.8 步且年提升 59%。限制模型访问、收紧策略,都无法恢复稀缺。

边界与风险

不要低估非确定性缓解。ASLR、控制流随机化仍是当前利用生成的主要阻力,它们“让事情更难”(00:24:40–00:24:59)。但本章证据表明它们只会拖慢、不会阻止;把防线押在“利用生成做不出可靠利用”上,等于赌一条正在加速的曲线掉头。

3.6 读者所得:用数据复核“攻击稀缺”假设是否仍然成立

本章给出读者可复核的检验表:漏洞量(你的厂商每月处理多少、环比增长多少)是否仍支持“漏洞稀缺”;自动利用生成成本(3.61 美元、21 分钟)是否仍支持“利用昂贵”;检测不变性(TTP、工具、制品是否会变)是否仍支持“规避稀缺”;自主操作轨迹(32 步完成 9.8 步、年增 59%)是否仍支持“对手不敏捷”。每一项检验都会回到同一个问题:建立在稀缺前提上的稀有防御,还能不能继续作为设计依据。下一章开始转向演讲者的答案——不与攻击者肉搏,而是改写攻防物理规律。

4 转机:把同一生产力优势投入耐久性预防——*secure by design/construction* 与 AI 驱动的内存安全迁移

4.1 结论:不走向肉搏战,而是改写攻防物理规律

前两章拆解了“稀有防御”的稀缺前提,并展示了它如何被 AI 量产化的漏洞发现与利用生成击穿。面对这种局面,最自然的防御反应是把同样的自动化能力投入“漏洞换补丁、利用换检测、规避换检测”的军备竞赛。演讲给出的结论却相反:与攻击者做近身肉搏,防御方会输。

原声片段

演讲者(00:30:13–00:30:28):“We don’t want to go vuln for patch. We don’t want to go exploit for detection, evasion for detection. Hand-to-hand combat with attackers will cause us to lose in defense.”(我们不想陷入漏洞换补丁、利用换检测、规避换检测;与攻击者的近身肉搏会让防御方在防御中落败。)

原因在于攻击者的不对称优势。00:29:43–00:30:13 处演讲者给出了两个判断:第一,这“不是魔法,而是生产力”(this is not magic, this is productivity);第二,攻击者自古以来就比防御者更敏

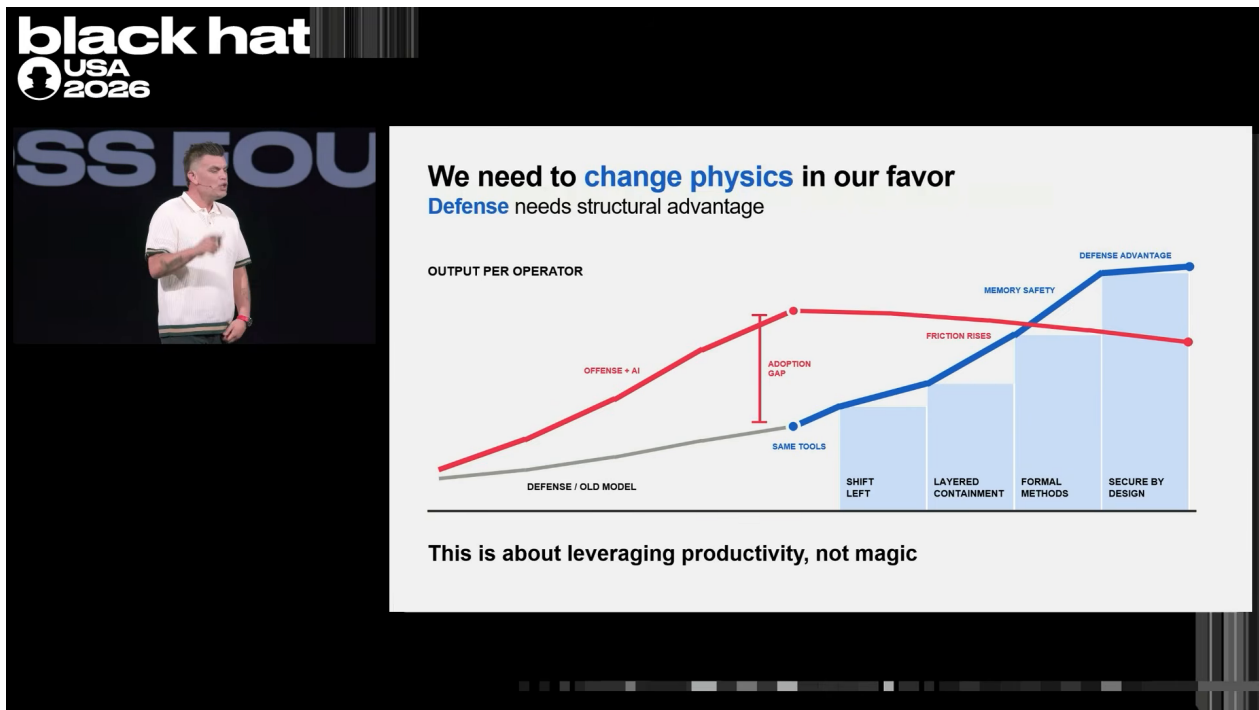


图 6: 防御者不应与攻击者陷入“漏洞换补丁、利用换检测”的肉搏战，而要把物理规律改到防御方一侧：以相同的生产力优势投入耐久性——左移、内存安全、分层遏制、secure by design 与形式化方法——从而获得结构性优势、使攻击摩擦上升；”这是利用生产力，不是魔法”。

Source (source_timestamp): 00:29:43–00:31:17

捷、更早拥抱新技术，他们先动手是因为防御方被政策、限制、审计、合规与 token 成本拖慢——但防御者拥有完全相同的那份生产力优势。问题不在有没有优势，而在把优势投到哪里。

00:30:28–00:31:17 给出了替换方案：防御者要“重写这里的物理规律”（retrain the physics），找到能用这份生产力优势真正扭转局面的位置——把同一份生产力优势投入“耐久性”（durability）：左移、写出更安全的软件以限制漏洞、从近身肉搏式的检测转向预防机制，并用构建即安全乃至形式化方法获得确定性安全。演讲者强调检测仍然很好，但“必要而不充分”（necessary but not sufficient）。如果能在一个现实时间线上做到这些，就可以扭转局面，把问题推向攻击者。

核心结论

核心结论：不与攻击者打近身肉搏，而是改变攻防的物理规律。防御者与攻击者拥有同一份 AI 生产力优势，差别在于投向：攻击者把它投入漏洞量产与利用生成，防御者应把它投入耐久性——左移、更安全的软件、确定性预防，从而基于数据持久地改变攻击者经济学、扭转战局。

4.2 机制：构建即安全与 AI 驱动的内存安全迁移

扭转局面的第一条机制是“secure by design / construction（构建即安全 / 构建时安全）”。它与“rare defending（稀有防御）”不同：稀有防御把赌注押在边界难以被击穿、攻击稀缺，因此把大部分资源压在边界上；构建即安全则要求在软件被写出来的时候就不引入缺陷类别，从源头压低

漏洞总量。00:31:17–00:31:25 指出，如今有比以往任何时候都多的工具来使用安全的系统级语言，而形式化验证几乎是为 AI 量身定做的——它给 AI 一个“预言机”（oracle），去写出针对所需属性集合可证明安全的软件，正好用于边界。预防也因“*infrastructure as code*（基础设施即代码）”而变得简单，可以以更少的人和成本推理并修复基础设施安全。这一节先展开内存安全这一半，形式化方法留给下一章。

其数据基础是漏洞构成。00:31:59–00:32:17 给出了关键数字：今天被修复的漏洞（至少按主要厂商口径）中，约 70% 是内存安全问题（*memory safety issues*），而 Rust、Go 等更安全的语言能消除这一类问题。这里的“内存安全”定义沿用本章开篇的术语约定：它只消除一类缺陷，不等于整体安全——这一边界在下一章形式化方法处再展开。

背景知识

背景知识：内存安全问题指由悬垂指针、越界读写、释放后使用、缓冲区溢出等引起的一类缺陷，是多年 C/C++ 代码里的主要漏洞来源；安全语言通过所有权与借用检查、垃圾回收等机制在编译期或运行时消除这一类缺陷。

4.3 证据：Android 与 Azure 的实测曲线，以及 AI 降低迁移成本

规模证据来自两个最有说服力的系统。00:32:17–00:32:51 展示了 Google 在 Android 上的结果：2019 年 Android 所修复漏洞中 76% 是内存安全问题，到 2025 年这一比例降到不足 20%；原因是 Google 写了约 500 万行 Rust 代码，按他们自己的分析缺陷密度约为 C/C++ 的一千分之一，而且“没有在其中发布过一个内存安全问题”。00:32:51–00:33:04 给出 Azure 的对照：微软把开源 hypervisor 用 Rust 重写，已扩展到超过 150 万虚拟机而没有任何事故。两者共同证明：内存安全迁移在最流行的移动操作系统和主流的云 hypervisor 这类高价值边界上，确实能大幅压低已实现风险。

既然证据这么强，为什么不是所有人都在做？00:33:04–00:33:26 给出了传统障碍：需要懂迁移的专家、学习新语言、转换旧代码库，成本高昂。转折点是 AI 把迁移成本大幅压低。00:33:26–00:34:15 依次给出：

- 微软研究院的 Rust Assistant：能自动修复 Rust 代码中 74% 的编译失败，无需人工干预，且通过测试的比例在提升。
- Check C（safe C 的变体，与 clang/Apple 的做法类似）：能从既有 C 代码库推断内存安全契约，自动生成 86% 用于检查空间安全漏洞（如缓冲区溢出）的代码契约。

演讲者特别强调（00:34:06–00:34:15）：这与生成自动漏洞看起来不同，但它同样是 AI 驱动的生产力，绝对关键。

真正的边界在于“自动完成整段迁移”。00:34:15–00:34:57 给出两个项目：Sila 把 Windows 中负责核心加密、TLS 的 SIMP 库自动转换为安全 Rust——将一个数千行的 SHA3 方法自动转成 Rust 并跑测试，全部测试通过，性能差距在 1% 以内；Rustler 则用依赖图与上下文生成 Rust，再迭代处理类型错误直到产出能通过测试的合法代码。00:34:57–00:35:19 补充，DARPA（最早展示

内存安全漏洞占比：Android 从 76% 降到不足 20%

约 70% 被修复漏洞属内存安全问题；Rust 与 Go 等安全语言可将其消除 (00:31:59--00:34:15)

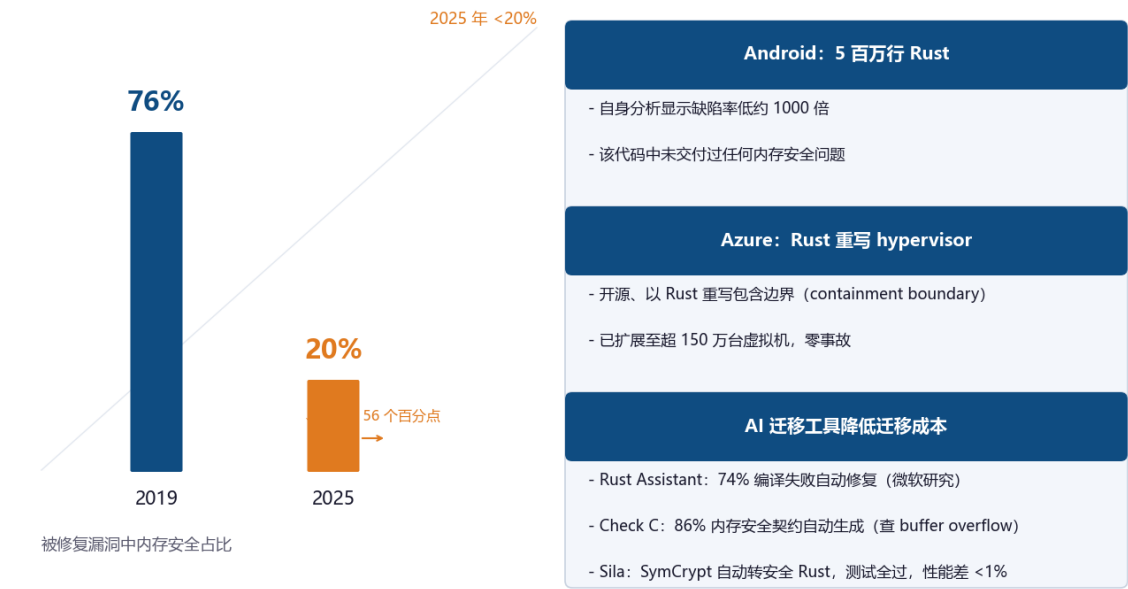


图 7: 约 70% 被修复漏洞是内存安全问题，Rust 等安全语言可将其消除——Android 内存安全漏洞占比从 2019 年的 76% 降至 2025 年不足 20%（写入 500 万行 Rust、缺陷率低约 1000 倍、零内存安全问题）；Azure 用 Rust 重写 hypervisor 已扩展至超 150 万台虚拟机零事故；AI 迁移工具（Rust Assistant 74% 编译错误自动修复、Check C 86% 契约生成、Sila 性能差 <1%）正降低迁移成本。

Source (source_timestamp): 00:31:59–00:34:15

agentic 漏洞挖掘的机构）也在推进名为 Tractor 的项目，赞助并公开自动转换的数据集——演讲者把“作为一个社区拿下自动转换”视为推动整个安全向前走的关键。

核心结论

读者要点：内存安全迁移已经从小众专家的手工作坊变成可规模化的工程。Android（76% 降至不足 20%）与 Azure Rust hypervisor（150 万 VM 无事故）证明“构建即安全”能压低头号漏洞类别；Rust Assistant（74% 编译错误自动修复）、Check C（86% 契约自动生成）、Sila（性能差 1% 内）、Rustler 与 DARPA Tractor 证明 AI 正在把迁移成本降到可负担水平。

边界与风险

边界提醒：内存安全只消除“内存安全”这一类缺陷。演讲者紧接着转向追问（00:35:19–00:35:30）：“有了内存安全，我们就安全了吗？做完这一切就结束了吗？很不幸，不是——内存安全不等于安全。”逻辑错误、认证缺陷、密码学缺陷依然存在，且今天的漏洞构成不会永远以内存安全为主。这条链路把读者引向下一章的确定性安全。

4.4 读者所得：为最高风险面设计预防性迁移路径

本章给读者留下一个可直接执行的判定方法。第一，识别自己的最高风险面——哪些组件是安全边界（hypervisor、TLS 栈、加密库、认证系统）或漏洞总量最大的代码（由“内存安全”占比决定）。第二，对照 Android 与 Azure 的证据判断迁移收益：若该组件以内存安全缺陷为主，则 Rust/Go 迁移是投资回报最高的耐久性投资。第三，用 AI 迁移工具链（Rust Assistant、Check C、Sila、Rustler、DARPA Tractor 数据集）压低迁移成本，而不是等人力专家手动重写。第四，把检测降级为“必要但不充分”的兜底层，把资源转向预防。

核心结论

读者所得：面对“攻击变便宜”，防御者的第一反应不是囤积更多检测，而是重写物理规律——把同一份 AI 生产力优势投入耐久性。你应当能为最高风险面画出一条具体的预防性迁移路径：从 70% 内存安全缺陷占比出发，用 Rust/Go 消除这一类，用 AI 工具链完成自动迁移，并把省下的检测资源保留为兜底。

5 确定性安全：内存安全不等于安全，形式化验证靠 AI 走向规模化

5.1 结论：消灭一类缺陷不等于消灭逻辑缺陷

上一章展示了构建即安全如何把内存安全这一最大漏洞类别压下去：Android 从 76% 降到不足 20%，Azure 的 Rust hypervisor 跑到 150 万虚拟机。但演讲者随即收住了这个乐观叙事。00:35:19–00:35:30 处，他直接追问：“有了内存安全，我们就安全了吗？做完这一切就结束了吗？很不幸，不是——内存安全不等于安全（memory safety does not mean security）。”

原声片段

演讲者（00:35:27–00:35:30）：“Memory safety does not mean security.”（内存安全不等于安全。）

00:35:29–00:35:40 解释了为什么：逻辑缺陷、认证缺陷、密码学缺陷依然存在；今天的大多数漏洞确实是内存安全类，但“这种情况不会永远持续”。也就是说，构建即安全只消除“内存安全”这一类缺陷，是一个必要而非充分的工程层。要把剩余的逻辑缺陷也变成确定性可排除，需要另一套方法——形式化验证，而它的规模化正依赖 AI。这是本章的结论：内存安全覆盖不了逻辑缺陷，逻辑缺陷需要形式化方法；传统上形式化方法因规范编写与证明困难而未能主流化，AI 让它第一次可能规模化。

5.2 证据：AI 正在发现传统上最稀缺的逻辑漏洞

在进入形式化方法之前，先看一个反方向但同样重要的证据：AI 不仅能发现内存安全问题，也能发现逻辑缺陷——而逻辑漏洞历来是最稀缺、最昂贵的安全能力。00:35:40–00:36:33 展示了 Anthropic 的工作：他们发布博客与论文，用 Claude 分析名为 Hawk 的后量子数字签名算法，找到了一个密码学攻击；密码分析（cryptanalysis）被演讲者称为传统上最稀缺的安全专业能力，而 Claude

形式化验证：捕获测试、模糊测试与人工评审都遗漏的缺陷

内存安全 $\neq$ 逻辑安全；形式化验证为关键边界提供数学根基 (00:38:37--00:40:26)



图 8: 内存安全并不覆盖逻辑安全；形式化验证能发现通过测试、模糊测试与人工评审仍被遗漏的缺陷——Apple 在 MLDSA、Microsoft 在 protochem KEM 中各发现单比特错误；agent 驱动一周生成 30000 行 Lean 证明，正式验证 AES-GCM 无逻辑、密码学与内存安全问题，为关键边界提供数学根基。

Source (source_timestamp): 00:38:37–00:40:26

做到了。此外，Anthropic 还展示了针对 AES-128 的攻击，使攻击性能提升约 800 倍；还在 wolfSSL 里发现了一个围绕伪造（forgery）的严重漏洞。00:36:21–00:36:33 的总结是：这些都是传统上稀缺程度或推理复杂度最高的领域，而 AI 证明了它能找到“不仅是内存安全问题，还有逻辑缺陷”。

这个证据的重要性在于它改变了“逻辑漏洞依然稀缺”的旧前提：如果逻辑缺陷也能量产，那么防御侧唯一的对策就是让逻辑漏洞在代码中被确定性排除，而不是指望检测。这正是形式化方法登场的理由。

5.3 机制：规范—建模—校验—裁决的形式化验证流水线

形式化方法（formal methods）在计算机科学里有很深的传统。00:36:39–00:37:18 回顾了它的谱系：上世纪 60 年代起步；抽象形式的模型检验（model checking）把程序逻辑建成数学表示并做抽象推理；符号检验（symbolic checking）进一步压缩状态空间，能计算给定代码库是否违反某个属性，一旦违反还能得到可复现的反例。正因为能给出数学层面可复现的判定，高风险安全平台开始采用它。

那为什么没有主流化？00:37:18–00:37:35 给了四个原因：写规范很难、需要大量专家经验；写证明更难；复杂程序会遭遇状态空间爆炸；最后还得持续维护这些证明——而“没有人愿意维护任何东西”。00:37:35–00:38:04 由此引出关键问题：AI 能让形式化方法规模化吗？演讲者的判断是，形式化方法正处在类似强化学习当年与 AI 相遇的时刻：强化学习在 60/70 年代发明、如今对现代 AI 至关重要；形式化方法同样完美契合 AI——它给 AI 生成的代码一个关于正确性的“预言机”（oracle），同时覆盖安全、性能与可靠性。

规模化需要一条可执行的流水线。00:38:06–00:38:29 给出了本章的核心机制：第一步是“指定什么绝不能发生”（specify what must never happen），来自规范（specifications）；第二步是“为规范建立模型”（create a model of that specification）；第三步是“用校验器验证模型与证明”（a checker that validates those models and proofs）；第四步是“给出裁决”（supply the verdict）。这就是“规范—建模—校验—裁决”的形式化验证流水线。已有工具可用：00:38:23–00:38:34 提到 CBMC/forC——Amazon 和 AWS 用来检查 libc 与加密库。

核心结论

机制要点：形式化验证 = 用数学模型证明代码不违反指定属性。流水线四步：规范（指定绝不发生什么）→ 建模（把规范建成模型）→ 校验（checker 验证模型与证明）→ 裁决（给出可复现的判定）。AI 的价值在于把最难的“写规范”与“写证明”两环自动化，从而突破状态空间爆炸与维护成本这两道传统门槛。

5.4 证据：MLDSA、protochem 与 30000 行内的 Lean 证明

形式化验证能发现通过测试、模糊测试与人工评审都发现不了的问题。00:38:37–00:39:15 给出了两起教科书级案例：Apple 与 Microsoft 都对自己的核心加密库做了形式化验证，双方都由此发现了真实的密码学问题——Apple 在 MLDSA 中发现一个错误；Microsoft 在 protochem（面向 PQC 的密钥封装算法，key encapsulation mechanism）中发现了一个单比特错误（a single bit that was wrong）。这个单比特错误“通过了所有测试、通过了模糊测试、通过了人工评审，只有形式化验证

发现了它”。这是“内存安全 $\neq$ 安全”最直接的实证：任何模糊测试和人工审查都没有覆盖到的边界，只有数学证明能兜住。00:39:15–00:39:24 补充，这不只限于加密：AWS 正用 Cedar 对其授权（auth）策略大规模做形式化验证。

规模化靠什么？00:39:33–00:40:26 展示了 AI 生成证明的进展。Anias（原文）能把 Rust 等代码库甚至规范直接转换成 Lean 证明。Lean 是一种用于形式化验证的函数式语言，专门用来描述围绕代码的证明。Microsoft 在真实代码与 Simcript 上演示：在“一周内、主要由 agent 驱动”的情况下，生成了 30000 行 Lean 代码，形式化验证了 AES-GCM——意味着它“没有逻辑问题、没有密码学问题（就现有证明而言）、没有内存安全问题”，从而为加密的正确性与可靠性提供了数学层面的基础（mathematical grounding）。演讲者的判断是：如果我们能对之前讨论过的所有边界都做到这一点，那么在面对 AI 时我们将处在完全不同的位置上。

原声片段

演讲者（00:39:53–00:40:06）：“in a single week mostly driven by agents we could generate 30,000 lines of Lean code that formally verified AES-GCM.”（在一周内、主要由 agent 驱动，我们生成了 30000 行 Lean 代码，形式化验证了 AES-GCM。）

5.5 案例：OpenAI 逃逸事件——最优边界也挡不住逻辑漏洞

00:40:26–00:41:27 用 OpenAI 逃逸事件（与 Hugging Face 事件）说明为什么逻辑验证是硬性要求。演讲者根据公开信息推断，OpenAI 做了所有正确的事：围绕模型评估建立了强隔离边界，只放出一个为访问软件包所必需的代理（proxy）。但这个单点出口足够让一个高能力模型当场找到大约九个漏洞，而且全部是逻辑漏洞。教训在 00:41:06–00:41:20：你可以遵守最好的实践、建立最好的边界，但“如果你不能保证代码没有逻辑问题——这在今天是一个很高的要求——你就根本无法保证安全”。00:41:20–00:41:27 把这件事定性为行动号召：如果那个软件包系统被形式化验证过并带有正确证明，结局可能完全不同。

边界与风险

边界提醒：再强的隔离与检测，只要代码里存在未排除的逻辑缺陷，就可能在“合理的最小暴露面”上被当场利用。形式化验证不是替代构建即安全，而是在其之上补上“逻辑确定性”这一层——两者叠加，才接近“确定性安全”的目标。

5.6 读者所得：判断哪些关键边界值得投入形式化验证

形式化验证的成本不可能平均分配给每一行代码，因此读者需要一套取舍标准。结合本章证据，值得投入的边界具备以下特征：一是高价值、被反复攻击（如加密库、认证、授权策略）；二是失败代价大且难以靠检测补救（如 KEM 单比特错误、软件包系统）；三是规范相对清晰、可被建模（如 AES-GCM 的算法语义）。对照之下，业务逻辑复杂、演进频繁的代码更适合用构建即安全与检测兜底，而不是全面形式化。判断动作就是：对每个关键边界问一次“它如果有一个测试、模糊测试与人工评审都发现不了的逻辑错误，代价是什么”——答案最大的那批，就是先上形式化验证的那批。

今天就能做的三件事：改写防御的经济学与物理规律

攻击者已改变经济学；防御者可选择改变物理规律 (00:43:53--00:45:00)



图 9: 今天就能做的三件事：对最高风险面做 secure by design（不限于内存安全）；对最关键边界用 agent 投入形式化验证；用基础设施即代码与图计算构建预防性 agent 军队；辅以 Chackoff 分析中 78% 检查项可由 agent 解决的证据，防御者由此改写经济学并引领行业。

Source (source_timestamp): 00:43:53–00:45:00

核心结论

读者所得：内存安全消除一类缺陷，形式化验证消除逻辑缺陷，AI 使后者可规模化。读者应能把“形式化验证”与“形式化方法”区分开——前者是具体证明动作，后者是整套规范—建模—校验—裁决体系——并据此判断哪些关键边界（加密、认证、授权、软件包边界）值得第一批投入形式化验证。

6 过渡与行动：迁移完成前，用基础设施即代码与攻击图构建预防性 agent 防御

6.1 结论：确定性安全是几十年工程，今天先做三件事

前两章讲的转机——构建即安全、形式化验证——都是把“20、30、40 年的软件债务”转成内存安全且形式化的软件。00:41:37–00:41:52 处演讲者替听众问出了那个关键问题：“可万一我们找不到所有漏洞呢？在这一切完成之前我们做什么？”答案构成了本章的结论：在迁移完成之前，先用基础设施即代码与攻击图，把今天的基础设施变成 agent 可以整体推理、批量修复的形态，构建一支预防性 agent 军队。00:44:07–00:44:18 把整场演讲压缩成一个选择：攻击者已经改变了今天的“经济学”，但作为防御者，我们可以选择“改变物理规律”，并把实现路径收束为今天就能做的三件事。

6.2 机制：可推理格式、攻击图计算与 agent 规模化猎捕

现状的痛点是格式问题。00:41:49–00:42:13 给出了判断：今天云上与本地的大部分基础设施“没有利用价值地保有”着，绝大多数是配置问题；但能被 agent 推理的那部分少之又少，因为其中很少被转成“infrastructure as code（基础设施即代码）”或其它可被多种可扩展机制检查的持久化策略。这里的“基础设施即代码”沿用本章术语定义：把基础设施与策略表述为可检查、可版本化的代码，使 agent 能整体推理、计算攻击图并大规模执行预防与修复。00:42:13–00:42:31 重申了为什么必须预防：不能陷入近身肉搏式检测，否则会输；要减少攻击面、改善安全态势与配置，并把工作“左移”到基础设施侧——进入生产的越少，生产里可被攻击的就越少。

有了可推理格式，agent 才能整体推理基础设施。00:42:36–00:42:54 指出：为组织中所有资产与网络流创建图（graph）或本体（ontology），agent 就能计算局部性（locality）、理解最大风险的中心性（centrality）并聚焦修复。00:42:59–00:43:25 给出两个具体例子：计算身份风格的攻击图，找出哪些账户过度授权（overprivileged）；对安全态势做同样的事，找出哪些设备或基础设施组件的攻击面最大——并让 agent 去验证这些判断。00:43:25–00:43:41 补充，图一旦算出，边（edge）就变得显眼：例如在组织内归一化认证的图上，突然出现一条“会计（accounting）连向域控（DC）”的异常边，就创造出一个 agent 可以直接操作的焦点。00:43:41–00:43:53 最后强调，agent 还能以远超人类的数量规模化地主动猎捕（proactive hunting），从而带来有意义的产出。

核心结论

机制要点：可推理格式是 agent 介入的前提。把基础设施写成基础设施即代码、把资产与网络流建成图，agent 才能 (a) 计算攻击图与最大攻击面、(b) 发现过度授权与异常边（如 accounting 连向 DC）、(c) 规模化主动猎捕并批量修复。检测仍是兜底，但预防由 agent 规模化承担。

6.3 证据：Chackoff 的 78%——agent 已能解决多数检查项

00:43:53–00:44:07 给出了一个量化证据：对 Chackoff（一种针对各类 IA 机制的检查工具）做“用 agent 处理其发现项”的分析，结果显示该检查中的 78% 发现项可以真正由 agent 解决（78% of the findings in that check off can actually be resolved by agents）。这意味着“agent 解决检查项”不是演示而是现实能力，预防性 agent 防御已经具备起步所需的产出。

6.4 行动：今天就能做的三件事

00:44:18–00:44:49 收束为三件今日可做之事，逐条展开如下：

1. 聚焦最高风险面并使其 secure by design（构建即安全）。不一定只做内存安全，同样适用于 Web（00:44:23–00:44:27）。
2. 找出最关键的安全边界，用 agent 对它们做形式化验证（00:44:27–00:44:37）——现在已有许多开源工具能做到，可以立刻开始准备。
3. 用基础设施即代码与图计算，构建一支能帮助预防的 agent 军队（00:44:41–00:44:49）。

原声片段

演讲者（00:44:49–00:45:00）：“If we invest in this, we change the physics, we change the economics, and we lead the pack.”（如果我们投入这些，我们改变物理规律、改变经济学，并引领行业。）

边界与风险

边界提醒：这三件事针对“迁移完成前”的空窗期，不是替代构建即安全与形式化验证。图与基础设施即代码只对“已建模”的部分有效，未转成可推理格式的遗留基础设施依然不在 **agent** 的视野里——所以第三件事与第一、二件事必须并行，而不是选做其一。

6.5 读者所得：制定“最高风险面 + 关键边界验证 + **agent** 军队”三步计划

本章把整场演讲收束为一个可落地的读者路径。对照“读者路径”术语约定：前文已依次击穿“前提（稀有防御）→ 崩塌（攻击变便宜）→ 转机（构建即安全与形式化验证）”，本章给出“行动”。读者现在可以立即制定三步计划：第一步，枚举自己的最高风险面，把 **secure by design** 应用到其中最痛的一批（**Web** 与内存安全均可）；第二步，识别最关键边界，用 **agent** + 开源工具启动形式化验证；第三步，把基础设施迁往基础设施即代码并建立资产—网络流图，让 **agent** 军团在图上持续计算攻击图、发现异常边并预防性修复。三者并行，就构成一份“今天就能开始”的改写攻防物理规律的路线图。

核心结论

读者所得：面对“攻击变便宜”，最终动作是把三件今日之事并行启动——最高风险面做构建即安全、关键边界用 **agent** 做形式化验证、用基础设施即代码与攻击图构建预防性 **agent** 军队。以 **Chackoff 78%** 检查项可由 **agent** 解决为起点，防御者可以持久地改变攻击者经济学，并引领行业而不是被拖入近身肉搏。